Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Самарский государственный технический университет»
(ФГБОУ ВО «СамГТУ»)

Кафедра «Высшая математика»

Оценка влияния выбора начального приближения сингулярного разложения

название работы

Курсовая работа

Выполнил студент
__1_ курса, 113М группы
Бекиш Егор Павлович

_____
подпись

Руководитель
Доцент, к. ф.-м. н.
Фадеева Оксана Владиславовна

_____
подпись

Работа защищена
«____» _____ 202__ г.

Оценка _____

Самара, 202__ г.

Оглавление

Введение **3**

1. ОПИСАНИЕ МЕТОДОВ **4**

1.1 Сингулярное разложение **4**

1.1.1 Нахождение правых сингулярных векторов **4**

1.1.2 Нахождение левых сингулярных векторов **5**

1.1.3 Нахождение сингулярных чисел **5**

1.2 Прямой метод **7**

1.3 Степенной метод **9**

1.4 Рандомизированный метод **10**

2. ОЦЕНКА МЕТОДОВ **11**

Заключение **19**

Литература **20**

Введение

Сингулярное разложение (SVD) [1] – один из важных методов в математика, который широко используется в прикладных задачах, таких как: компьютерное зрение, обработка изображений, обработка данных и т.п.

Сам по себе метод, с точки зрения расчета, не имеет никаких сложностей, но для решение вышеописанных задач, для которых необходима ЭВМ, возникают определенные параметры и критерии оценивания алгоритма. В частности – начальное приближение при решении СЛАУ при расчетах матриц, которые используется для разложения. Для этого существуют численные методы, которые предлагают алгоритмы для итеративных методов вычисления SVD, такие как:

- Прямые [2].
- Степенные (итеративные) [3].
- Рандомизированные [4].
- Проекционные.

В частности, для удобства, будут рассмотрены те методы, которые уже реализованы в фреймворках языка программирования Python [5].

1. ОПИСАНИЕ МЕТОДОВ

1.1 Сингулярное разложение

В общем случае всегда можно применить стандартную форму метода сингулярного разложения. Суть в том, чтобы уменьшить размер полученных данных, не потеряв при этом их качество. Разложение имеет следующий алгебраический вид:

$$A = U\Sigma V \tag{1.1}$$

где A – исходная матрица $m \times n$.

U – ортогональная матрица $m \times m$, образующая левые сингулярные векторы.

Σ – диагональная матрица $m \times n$, образующая сингулярные неотрицательные числа.

V – ортогональная матрица $n \times n$, образующая правые сингулярные векторы.

1.1.1 Нахождение правых сингулярных векторов

Для вычисления правых сингулярных векторов необходимо найти матрицу A_r следующего алгебраического вида:

$$A_r = A^T A \tag{1.2}$$

где A^T – транспонированная исходная матрица.

Для полученной матрицы A_r размера $n \times n$ необходимо вычислить характеристический многочлен, чтобы найти собственные значения, которые помогут получить правосторонние сингулярные векторы. Характеристический многочлен можно вычислить любым удобным способом, как в стандартом виде $|A - \lambda I| = 0$, так и с помощью инвариантов. В конечном итоге, необходимо нормировать каждый вектор, выписать их по столбцам, нормировать и транспонировать полученную матрицу. Таким образом получим правосторонние сингулярные векторы. Алгебраический вид, следующий:

$$V = (V_1 \quad V_2 \quad \cdots \quad V_n)^T \tag{1.3}$$

1.1.2 Нахождение левых сингулярных векторов

Левосторонние сингулярные векторы получается точно таким же образом, только нахождение матрицы A_l размера $m \times m$ отличается и имеет следующий алгебраический вид:

$$A_l = AA^T \tag{1.4}$$

Конечно, придется снова вычислить характеристический многочлен для матрицы большего размера, но можно этого не делать, т.к. при расчетах получится так, что собственные значения повторяться и добавятся только нули. После вычисления собственных векторов, так же нормируем каждый вектор и выписываем по столбцам, но уже без конечного транспонирования. Таким образом мы получим левосторонние сингулярные векторы. Алгебраический вид, следующий:

$$U = (U_1 \quad U_2 \quad \cdots \quad U_m) \tag{1.5}$$

1.1.3 Нахождение сингулярных чисел

В процессе нахождения, как правосторонних, так и левосторонних сингулярных векторов у нас были найдены их собственные числа. Для получения сингулярных чисел – необходимо отсортировать полученные собственные числа в порядке убывания и извлечь из каждого квадратный корень. Тем самым, получится матрица $m \times n$, где по главной диагонали будут расположены сингулярные числа. Алгебраический вид, следующий:

$$\Sigma = \begin{pmatrix} \sqrt{\delta_1} & 0 & \cdots & 0 \\ 0 & \sqrt{\delta_2} & \cdots & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & \cdots & \sqrt{\delta_n} \\ 0 & 0 & \cdots & 0 \end{pmatrix} \tag{1.6}$$

где δ_i – собственные числа.

В качестве наглядного примера рассмотрим матрицу 3×2.

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 2 \end{pmatrix}$$

Рассчитаем правостороннею и левостороннею матрицы.

$$A_r = A^T A = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 2 \end{pmatrix} = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}$$

$$A_l = AA^T = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 2 \\ 2 & 2 & 8 \end{pmatrix}$$

Найдем собственные значения для A_r.

$$|A_r - \lambda I| = 0 \tag{1.7}$$

$$\begin{vmatrix} 5-\lambda & 4 \\ 4 & 5-\lambda \end{vmatrix} = (5-\lambda)^2 - 16 = \lambda^2 - 10\lambda + 9 = (\lambda - 9)(\lambda - 1)$$

Далее составляем СЛАУ для каждого собственного числа вида:

$$A_r x = \lambda_i x \to A_r x - \lambda_i x = 0 \to (A_r - \lambda_i)x = 0 \tag{1.8}$$

Рассмотрим пример нахождения собственного вектора для $\lambda = 9$:

$$\begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix} - \begin{pmatrix} 9 & 0 \\ 0 & 9 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0 \to \begin{pmatrix} -4 & 4 \\ 4 & -4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0$$

$$V_{\lambda=9} = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$$

Итоговая правосторонняя матрица имеет следующий вид:

$$V = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$$

Проделав те же самые действия с добавлением собственного числа 0, получаем левостороннею матрицу вида:

$$U = \begin{pmatrix} 1/\sqrt{18} & 1/\sqrt{2} & 2/3 \\ 1/\sqrt{18} & -1/\sqrt{2} & 2/3 \\ 4/\sqrt{18} & 0 & -1/3 \end{pmatrix}$$

Так же составим матрицу сингулярных чисел.

$$\Sigma = \begin{pmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$$

Выполним проверку, чтобы убедится в том, что у нас верное направление и матрицы равны.

$$U\Sigma V = \begin{pmatrix} 1/\sqrt{18} & 1/\sqrt{2} & 2/3 \\ 1/\sqrt{18} & -1/\sqrt{2} & 2/3 \\ 4/\sqrt{18} & 0 & -1/3 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix} =$$

$$= \begin{pmatrix} 3/\sqrt{18} & 1/\sqrt{2} \\ 3/\sqrt{18} & -1/\sqrt{2} \\ 12/\sqrt{18} & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 2 \end{pmatrix}$$

1.2 Прямой метод

В стандартной форме, как можно было заметить, при вычислениях не было ни начального приближения, не его оценки. Поэтому, стоит отметить, что при реализации алгоритма с помощью ЭВМ необходимы критерии остановки из-за особенностей хранения значений в языках программирования. В основе всегда лежат два критерия остановки: приближение и количество итераций. Для данного метода необходимы следующие алгоритмы для реализации и разложение:

- Ортогонализация Грамма-Шмидта [6].
- QR-разложение [7].

Порядок действий в ортогонализации:

1. $L = LB(a_1, \cdots, a_n) \to L = LB(b_1, \cdots, b_n) \Longrightarrow b_i \perp b_j \; \forall \; i \neq j$
2. $b_1 = a_1$

   $b_2 = a_2 - Pr_{b_1} a_2$

   $b_3 = a_3 - Pr_{b_1} a_3 - Pr_{b_2} a_3$

   …

   $b_n = a_n - \sum_{i=1}^{n} Pr_{b_i} a_n$
3. Нормируем полученные вектора.

Порядок действий в алгоритме разложения:

1. $A_0 = A$ – начальные собственные значения.

2. Ортогонализация матрицы $A_k \to Q_k$.

3. Получение верхней треугольной матрицы $R_k = Q_k^T A_k$.

4. $A_{k+1} = R_k Q_k$, если $\|A_{k+1} - A_k\| < \varepsilon \to diag(A)$, иначе возвращаемся на действие (2).

Собственные векторы получаются путем перемножения между собой матриц Q_k и каждый столбец умножается на свое собственное значение, такую матрицу обозначим за $\tilde{Q}_k$. Таким образом, получаем матрицы сингулярных значений и векторов.

В качестве примера, рассмотрим матрицу размера 3×2 на одной итерации. Действия получения правосторонней (1.2) и левосторонней (1.4) остаются идентичными.

$$A = \begin{pmatrix} 2 & 6 \\ 5 & 3 \\ -4 & 0 \end{pmatrix} \tag{1.9}$$

$$A_l = \begin{pmatrix} 45 & 27 \\ 27 & 45 \end{pmatrix} \; A_r = \begin{pmatrix} 40 & 28 & -8 \\ 28 & 34 & -20 \\ -8 & -20 & 16 \end{pmatrix}$$

Ортогонализируем матрицы A_l, A_r и получаем необходимые матрицы:

$$Q_0^r = \begin{pmatrix} 0.86 & -0.51 \\ 0.51 & 0.86 \end{pmatrix} \; R_0^r = \begin{pmatrix} 5.25 \times 10^1 & 4.63 \times 10^1 \\ 2.55 \times 10^{-15} & 2.47 \times 10^1 \end{pmatrix}$$

$$A_1^r = \begin{pmatrix} 68.82 & 21.18 \end{pmatrix} \; \tilde{Q}_1^r = \begin{pmatrix} 59.02 & 35.41 \\ -10.90 & 18.16 \end{pmatrix}$$

$$Q_0^l = \begin{pmatrix} 0.81 & -0.49 & 0.20 \\ 0.57 & 0.49 & -0.78 \\ -0.16 & -0.73 & 0.59 \end{pmatrix}$$

$$R_0^l \begin{pmatrix} 4.95 \times 10^1 & 4.51 \times 10^1 & -2.04 \times 10^1 \\ -2.22 \times 10^{-14} & 1.75 \times 10^1 & -1.75 \times 10^1 \\ -1.88 \times 10^1 & -3.29 \times 10^1 & 2.35 \times 10^1 \end{pmatrix}$$

$$\tilde{Q}_1^l = \begin{pmatrix} 55.64 & 38.95 & -11.13 \\ -17.46 & 17.46 & -26.19 \\ 4.15 & -16.61 & 12.46 \end{pmatrix}$$

Выполнив условия остановки или же пройде определенное количество итераций получаем следующие значения:

$$A = (72 \quad 18) \rightarrow \Sigma = \begin{pmatrix} \sqrt{72} & 0 \\ 0 & \sqrt{18} \\ 0 & 0 \end{pmatrix}$$

$$\tilde{Q}_r = V = \begin{pmatrix} 50.91 & 50.91 \\ -12.73 & 12.73 \end{pmatrix}$$

$$\tilde{Q}_l = U = \begin{pmatrix} -7.08 \times 10^{15} & 6.44 \times 10^{16} & -5.52 \times 10^{16} \\ -9.35 \times 10^{6} & 2.04 \times 10^{6} & -5.68 \times 10^{6} \\ -6.45 \times 10^{-10} & 5.87 \times 10^{-9} & -5.03 \times 10^{-9} \end{pmatrix}$$

$$U\Sigma V = \begin{pmatrix} -6.54 \times 10^{18} & 4.20 \times 10^{17} \\ -4.15 \times 10^{9} & -3.93 \times 10^{9} \\ -5.96 \times 10^{-7} & 3.83 \times 10^{-8} \end{pmatrix}$$

1.3 Степенной метод

Как можно было заметить, предыдущий метод отыскивает абсолютно все значения для разложения, что может эффективно работать на маленьких размерах матрицы, где в конечном итоге будет показывать свою эффективность, но когда размерность будет значительно большой – хранить и использовать абсолютно все найденные значения не будет иметь никакого смысла, т.к. уже выяснили, главная задача – сократить данные, не потеряв при этом важные компоненты. Поэтому, если размеры велики, то для этого можно использовать степенные методы, которые не вычисляют абсолютно все сингулярные числа, а только те, которые несут в себе наибольшею значимость.

Алгоритм имеет следующие этапы:

1. Генерируется случайны вектор b_0.
2. Вычисляется собственный вектор $b_{k+1} = \frac{Ab_k}{\|Ab_k\|}$
3. Вычисляется собственное значение $\mu_k = \frac{\langle b_k, Ab_k \rangle}{\langle b_k, b_k \rangle}$
4. Если $\|\mu_{k+1} - \mu_k\| < \varepsilon \rightarrow \mu = \mu_{k+1}; b = b_{k+1}$, иначе возвращаемся на действие (2).

Используя данный алгоритм на матрице (1.9), получаем следующие результат:

$$\lambda = 71.99999998095\ U = \begin{pmatrix} 0.666 \\ 0.666 \\ -0.333 \end{pmatrix} V = \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix}$$

Как можно заметить это напрямую отображает, что это схоже с понижениеv ранга матрицу при помощи SVD, следовательно, если перепишем в виде λUV, то можно получить идентичный результат, но уже с меньшем объемом данных.

1.4 Рандомизированный метод

Обращая внимания на вышеописанные методы, то в каждом есть как свои минуты, так и свои плюсы, тем самым, при решении задачи придется всегда проводить небольшой анализ для выбора лучшего метода, учитывать все за и против, что может повлечь за собой неоднозначность. Поэтому, был создан гибридный вариант, который может решать проблему памяти хранения, так и точности вычисления. Алгоритм Халко–Мартинассона–Троппа, который за счет случайной гауссовской матрицы проецирует части основной матрицы на ее пространства с меньшим размером путем ранжирования, получая меньшую матрицу, к которой можно применить стандартный SVD.

Этапы алгоритма, следующие:

1. Задается k – количество сингулярных чисел, p – параметр передискретизации.
2. Генерируется гауссовская матрица Ω размера $n \times k + p$.
3. Строится проекция $Y = A\Omega$.
4. Ортогонализация QR–алгоритмом.
5. Проецирование $B = Q^T A$.
6. Применение SVD к B.

2. ОЦЕНКА МЕТОДОВ

Выше были описаны основные методы, которые лежат в основе, как идеи, так и реализации, современных методов. Описать абсолютное все довольно трудно, но сравнить их и оценить, используя фреймворки языка Python, который направлен и предназначен на задачи Data Science, Machine Learning и т.п. можно. В качестве библиотек были использованы следующие:

- Scipy [8].
- Sklearn [9].
- Torch [10].
- Tensorflow [11].

Для реализации и демонстрации возьмем снимок из фреймворка Skimage [12]. На его основе будет использоваться разложение и реконструкция изображения (рис. 2.1).

Рисунок 2.1 – Исходное изображение

Применив варианты методов без критериев оценки, вариантов ортогонализации и т.п., которые представлены в таблице 2.1, получили восстановленные изображения с разным количеством компонент, как показано на рисунке 2.2.

Рисунок 2.2 – Реконструкция изображения без критериев оценки

Отдельно разберем реализацию с выбором определенных критериев для sklearn.decomposition.TruncatedSVD и scipy.sparse.linalg.svds, в которых уже можно выбрать метод ортогонализации, алгоритм разложения и т.п. Результаты для каждого варианта будут представлены в таблицах 2.2 – 2.3. На рисунках 2.3 – 2.4 изображены реконструированные изображения вариантов реализации.

Со всей реализацией можно ознакомится в репозитории GitHub [13].

Рисунок 2.3 – Реконструкция изображения с помощью sklearn.decomposition.TruncatedSVD

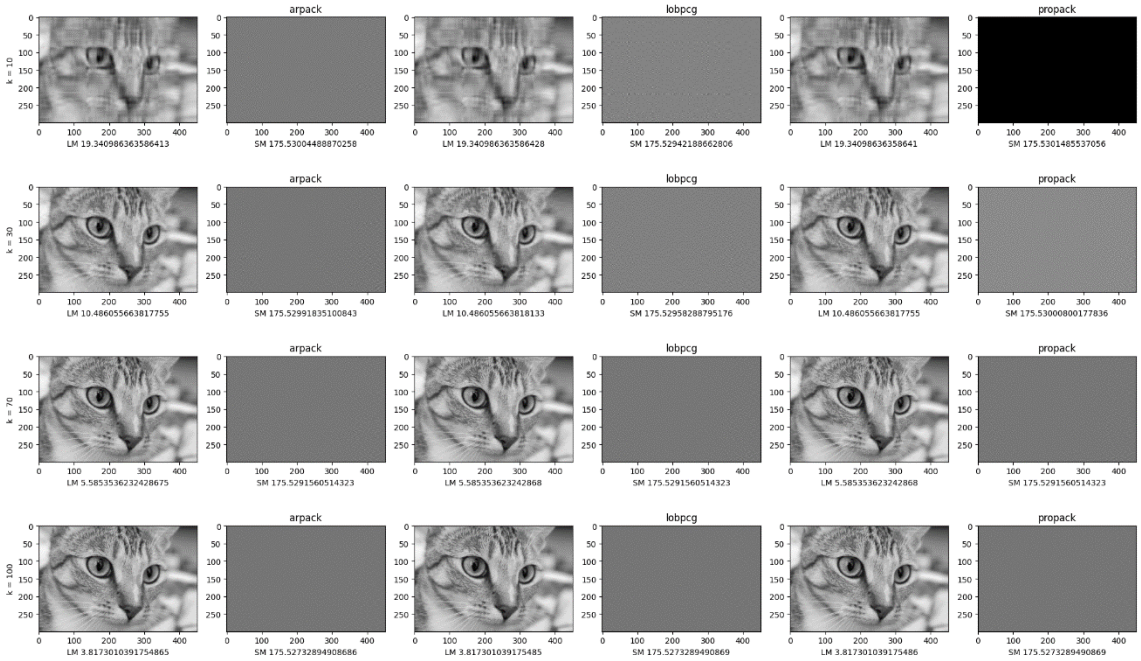

Рисунок 2.4 – Реконструкция изображения с помощью scipy.sparse.linalg.svds

Таблица 2.1 – Сравнение метод без критериев остановки

Метод \ Количество компонент	k = 5	k = 15	k = 30	k = 50	k = 70	k = 100	k = 200
np.linalg.svd	26,59612	15,73733	10,48606	7,414381	5,585354	3,817301	0,994909
scipy.linalg.svd	26,59612	15,73733	10,48606	7,414381	5,585354	3,817301	0,994909
sklearn.decomposition.randomized_svd	26,59612	15,73733	10,48808	7,420647	5,591879	3,823166	0,997105
sklearn.utils.extmath.randomized_svd	26,59612	15,73733	10,48825	7,41828	5,592023	3,823171	0,997939
torch.linalg.svd	26,59612	15,73733	10,48606	7,414381	5,585354	3,817301	0,994909

Таблица 2.2 – Результаты с критериями TruncatedSVD

Количество компонент	Алгоритм	Передискретизация	Ортогонализация	Ошибка
10	arpack	5	OR	19,34099
10	arpack	5	LU	19,34099
10	arpack	10	OR	19,34099
10	arpack	10	LU	19,34099
10	arpack	20	OR	19,34099
10	arpack	20	LU	19,34099
10	randomized	5	OR	19,35562

Продолжение таблицы 2.2

10	randomized	5	LU	19,34106
10	randomized	10	OR	19,34957
10	randomized	10	LU	19,34099
10	randomized	20	OR	19,34638
10	randomized	20	LU	19,34099
30	arpack	5	OR	10,48606
30	arpack	5	LU	10,48606
30	arpack	10	OR	10,48606
30	arpack	10	LU	10,48606
30	arpack	20	OR	10,48606
30	arpack	20	LU	10,48606
30	randomized	5	OR	17,49179
30	randomized	5	LU	10,49652
30	randomized	10	OR	16,71692
30	randomized	10	LU	10,48653
30	randomized	20	OR	16,45156
30	randomized	20	LU	10,48606

Продолжение таблицы 2.2

70	arpack	5	OR	5,585354
70	arpack	5	LU	5,585354
70	arpack	10	OR	5,585354
70	arpack	10	LU	5,585354
70	arpack	20	OR	5,585354
70	arpack	20	LU	5,585354
70	randomized	5	OR	15,7039
70	randomized	5	LU	5,594693
70	randomized	10	OR	15,59892
70	randomized	10	LU	5,587789
70	randomized	20	OR	15,20386
70	randomized	20	LU	5,585499
100	arpack	5	OR	3,817301
100	arpack	5	LU	3,817301
100	arpack	10	OR	3,817301
100	arpack	10	LU	3,817301
100	arpack	20	OR	3,817301

Продолжение таблицы 2.2

100	arpack	20	LU	3,817301
100	randomized	5	OR	14,4876
100	randomized	5	LU	3,829433
100	randomized	10	OR	14,25734
100	randomized	10	LU	3,8202
100	randomized	20	OR	13,78879
100	randomized	20	LU	3,817764

Таблица 2.3 – Результаты с критериями SVDS

Количество компонент	Алгоритм	Сингулярные значения	Ошибка
10	arpack	LM	19,34099
10	arpack	SM	175,53
10	lobpcg	LM	19,34099
10	lobpcg	SM	175,5295
10	propack	LM	19,34099
10	propack	SM	175,5301
30	arpack	LM	10,48606
30	arpack	SM	175,5299

Продолжение таблицы 2.3

30	lobpcg	LM	10,48606
30	lobpcg	SM	175,5296
30	propack	LM	10,48606
30	propack	SM	175,53
70	arpack	LM	5,585354
70	arpack	SM	175,5292
70	lobpcg	LM	5,585354
70	lobpcg	SM	175,5292
70	propack	LM	5,585354
70	propack	SM	175,5292
100	arpack	LM	3,817301
100	arpack	SM	175,5273
100	lobpcg	LM	3,817301
100	lobpcg	SM	175,5273
100	propack	LM	3,817301
100	propack	SM	175,5273

Заключение

В рамках данной курсовой работы было проведено исследование методов сингулярного разложения и оценено влияние выбора начального приближения и критериев остановки на точность вычислений.

В ходе анализа выявлено, что прямой метод эффективен для матриц малой размерности, однако требует значительных вычислительных ресурсов при росте объема данных. Степенной метод, напротив, позволяет находить лишь наиболее значимые сингулярные числа, что существенно экономит память, но может приводить к потере части информации. Рандомизированный метод представляет собой компромиссное решение, сочетающее преимущества обоих подходов.

Эксперименты по реконструкции изображения показали, что стандартные реализации демонстрируют идентичные результаты при отсутствии дополнительных критериев оценки. Анализ алгоритмов с настраиваемыми параметрами выявил, что ошибка восстановления напрямую зависит от комбинации выбираемых параметров: алгоритма разложения метода ортогонализации и параметра передискретизации.

Полученные результаты наглядно демонстрируют, что выбор начального приближения и параметров алгоритма критически важен для достижения баланса между точностью вычислений и объемом используемой памяти, что необходимо учитывать при решении прикладных задач обработки данных.

Литература

1. SVD [Электронный ресурс] // Wikipedia, 2026, URL: https://en.wikipedia.org/wiki/Singular_value_decomposition. (Дата обращения: 24.04.2026)
2. Numerical linear algebra [Электронный ресурс] // Wikipedia, 2026, URL: https://en.wikipedia.org/wiki/Numerical_linear_algebra. (Дата обращения: 24.04.2026)
3. Power iteration [Электронный ресурс] // Wikipedia, 2026, URL: https://en.wikipedia.org/wiki/Power_iteration. (Дата обращения: 24.04.2026)
4. Randomized algorithms for approximating the range of a matrix [Электронный ресурс] // Colorado University, 2012. URL: https://www.colorado.edu/2012_halko_dissertation.pdf (Дата обращения: 24.04.2026)
5. Python [Электронный ресурс] // Python, 1991. URL: https://www.python.org/. (Дата обращения: 24.04.2026)
6. Gram–Schmidt process [Электронный ресурс] // Wikipedia, 2026. URL: https://en.wikipedia.org/wiki/Gram%E2%80%93Schmidt_process. (Дата обращен ия: 24.04.2026)
7. QR-algorithm [Электронный ресурс] // Wikipedia, 2026. URL: https://en.wikipedia.org/wiki/QR_algorithm. (Дата обращения: 24.04.2026)
8. Scipy [Электронный ресурс] // Scipy, 2001. URL: https://scipy.org/. (Дата обращения: 24.04.2026)
9. Sklearn [Электронный ресурс] // Scikit-learn, 2007. URL: https://scikit-learn.org/stable/. (Дата обращения: 24.04.2026)
10. Torch [Электронный ресурс] // Torch, 2002. URL: https://pytorch.org/. (Дата обращения: 24.04.2026)
11. TensorFlow [Электронный ресурс] // Tensorflow, 2015. URL: https://www.tensorflow.org/. (Дата обращения: 24.04.2026)
12. Skimage [Электронный ресурс] // Scikit-image, 2009. URL: https://scikit-image.org/. (Дата обращения: 24.04.2026)

13. Реализация и исследования SVD [Электронный ресурс] // GitHub, 2025. URL: https://github.com/egorbeckish. (Дата обращения: 24.04.2026)